Rust Blockchain

Rust Blockchain

A Full-Stack Implementation Guide

Featuring Tokio, Axum, Iced, Tauri 2, Docker, and Kubernetes

Bill Kunyiha

First Edition

Rust Blockchain: A Full-Stack Implementation Guide

First Edition

Copyright © 2026 Bill Kunyiha. All rights reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the author, except for brief quotations in reviews and certain noncommercial uses permitted by copyright law.

ISBN: 979-8-9954311-0-7

Published by the author.

Source code repository:
https://github.com/bkunyiha/rust-blockchain

To the contributors behind Rust, Bitcoin Core, and the countless open-source projects that made this work possible. You proved that trust can be built in code.

Contents

Contents ix

1 Visual Style Sheet 1
- 1.1 Section Heading (Level 2) 1
- 1.2 Code Listings 2
- 1.3 Mathematics 2
- 1.4 Tables 3
- 1.5 Callout Boxes 3
- 1.6 Lists 5
- 1.7 Additional Elements 5

I Foundations & Core Implementation 9

2 Chain: Domain Model 13
- 2.1 Whitepaper Term → Rust Type Map 14
- 2.2 Domain Object Diagrams 14
- 2.3 Key Code Reading Checklist 15
- 2.4 Step-by-Step Code Walkthrough 15

Acronyms 19

Glossary 21

Index 23

Colophon 25

List of Listings

1 Block structure definition (`block.rs`) 2
2 Block and BlockHeader struct layout (`block.rs`) 16
3 `Block::new_block` — creating and mining a block 16
4 Transaction, TXInput, and TXOutput structs (`transaction.rs`) 17
5 UTXOSet struct (`utxo_set.rs`) 18

1

Visual Style Sheet

The root problem with conventional currency is all the trust that's required to make it work.

—Satoshi Nakamoto, 2009

In this chapter

1. Section Heading (Level 2)
2. Code Listings
3. Mathematics
4. Tables
5. Callout Boxes
6. Lists
7. Additional Elements

THIS page demonstrates every visual element used in the book. It serves as a design reference during the LaTeX conversion and ensures that all typographic choices work harmoniously together.

1.1 Section Heading (Level 2)

Body text in TeX Gyre Pagella (Palatino family) at 11pt with 1.05 line spread. Microtype provides character protrusion and font expansion for optically

even margins. Inline code appears as `BlockHeader` or `Vec<Transaction>` — note the subtle background pill that makes code pop from prose. Cross-references use Chapter 1.

Cross-refs are clickable in PDF

A **UTXO** (Unspent Transaction Output) is the fundamental unit of spendable value. First use of a key term prints bold and adds an index entry automatically.

Subsection Heading (Level 3)

Subsections use TeX Gyre Heros (Helvetica family) in the dark teal accent color.

Subsubsection Heading (Level 4)

The deepest heading level, still sans-serif and colored.

1.2 Code Listings

Listing 1: Block structure definition (`block.rs`)

1. The header contains only the fields needed for proof-of-work hashing and chain linking.
2. A block pairs its header with a vector of transactions, keeping structure and payload separate.

A code listing without line numbers (for shell commands):

```
cargo build --release
cargo test -- --nocapture
docker compose up -d
```

1.3 Mathematics

Inline math: the probability that an attacker catches up from z blocks behind is $q_z = (q/p)^z$ where p is the probability of an honest node finding the next block and $q = 1 - p$.

1.4 Tables

Table 1.1: Whitepaper concept to Rust type mapping

Whitepaper Concept	Rust Type	Chapter
Block header	`BlockHeader`	9
Transaction	`Transaction`	9
Input (spend)	`TXInput`	9
Output (new coin)	`TXOutput`	9
UTXO set	`UTXOSet`	9

1.5 Callout Boxes

All 16 canonical callout types:

▷ **Note**

This is a **Note** callout. Used for supplementary information that clarifies or expands on the main text.

⋆ **Tip**

This is a **Tip** callout. Practical advice the reader can apply immediately.

▲ **Warning**

This is a **Warning** callout. Highlights a common mistake or non-obvious behavior.

♦ **Important**

This is an **Important** callout. Critical information the reader must not overlook.

i

Methods involved `BlockchainService::add_block()`, `UTXOSet::update()`, `ProofOfWork::run()`

i

Source `bitcoin/src/chain/blockchain_service.rs` — BlockchainService

▷ **What you will learn in this chapter**

- How blocks link to form a chain via hash pointers
- The UTXO model for tracking spendable outputs
- How proof-of-work secures the chain

●●● **Prerequisites**

- Chapters 6–8 (Primitives, Utilities, Cryptography)
- Basic familiarity with hash functions and digital signatures

▷

See it in action Run `cargo test test_new_block` to see block creation and PoW in action.

▷

Implementation example The `add_block()` method validates the block, updates the chain tip, and triggers a UTXO set refresh.

↪

See the full implementation `bitcoin/src/chain/blockchain_service.rs` — full source with inline comments.

↪

Companion Chapter Complete source code is available in Chapter 9A: Code Listings. In the print edition, these listings appear in the Appendix: Source Reference.

i

Scope This section covers the domain model only. State management, persistence, and consensus are covered in Sections 9.2–9.7.

✓ Checkpoint

At this point you should understand: (1) why blocks contain a `pre_block_hash` field, (2) how transactions encode the UTXO worldview, and (3) where the signature lives in our model.

⌇ Troubleshooting

If `cargo test` fails with "database locked," ensure no other node instance is running. The Sled database uses exclusive file locks.

▷

Framework Comparison Iced uses the MVU (Model–View–Update) pattern; Tauri splits the Rust backend from a React/TypeScript frontend.

1.6 Lists

Bulleted list:

- `pre_block_hash`: link to the previous block (whitepaper §3/§4)
- `nonce` + `hash`: PoW search result (whitepaper §4)
- `timestamp`: ordering signal; also part of the PoW header bytes

Numbered list:

1. Define what a transaction *is* by reading the struct fields.
2. Show how a spending transaction is *constructed* in code.
3. Trace the UTXO selection, signing, and broadcast flow.

* * *

1.7 Additional Elements

The scene break above (`\scenbreak`) creates a visual pause between major topics within a chapter.

The `\keyterm` command marks first definitions: a **block header** is the metadata that identifies a block and links it to the previous one. Terms are indexed automatically.

Epigraphs appear at the start of each chapter (see the Satoshi quote at the top of this chapter). They set the tone without interrupting the technical flow.

Drop caps (`\lettrine`) open each chapter's first paragraph with a large decorative initial in the brand color, a hallmark of premium book typography.

Tips are green-accented

Margin notes appear in the outer margin for quick cross-references and asides without interrupting flow.

The `\cmark` command creates numbered annotation circles for code walkthrough explanations (see Listing 1).

> *"The UTXO set is derived state—not history. It answers one question: what is spendable right now?"*

▷ Deep Dive: Why we chose Sled over LevelDB

Our implementation uses the Sled embedded database instead of LevelDB (which Bitcoin Core uses). Sled is written in pure Rust, provides lock-free concurrent access, and integrates naturally with Rust's ownership model. The trade-off is a larger on-disk footprint and less battle-tested production history. For a learning project, the ergonomic benefits outweigh these costs.

A tutorial walkthrough with distinct step badges:

Step 1 **Clone the repository**

Run `git clone https://github.com/bkunyiha/rust-blockchain` to get a local copy of the project.

Step 2 **Build in release mode**

Run `cargo build -release` from the project root. This compiles with optimizations enabled.

Step 3 **Run the test suite**

Verify everything works: `cargo test - -nocapture`.

Footnotes[1] use brand-colored markers for visual consistency with the rest of the design system.

Table 1.2: API Reference: `BlockchainService`

Method	Parameters	Returns	Description
`add_block`	`block: Block`	`Result<()>`	Validates and appends a block to the chain
`get_tip`	`--`	`Option<Block>`	Returns the current chain tip
`get_block`	`hash: &str`	`Option<Block>`	Retrieves a block by hash

Key Takeaways

- ✓ All visual elements render correctly in Crown Quarto format
- ✓ Callout boxes have icon prefixes for instant recognition
- ✓ Code listings support annotation callouts with numbered circles
- ✓ Pull quotes, deep dives, and step environments add variety
- ✓ Tables use alternating row shading; API tables have a standard format
- ✓ Drop caps, margin notes, footnotes, and epigraphs add polish
- ✓ A colophon page documents the book's production details

[1] Like this one—note the rust-orange marker and thin brand-colored separator rule above.

Part I

Foundations & Core Implementation

Chapters 1–8 build your blockchain from first principles: cryptographic primitives, the UTXO transaction model, proof-of-work consensus, peer-to-peer networking, and a persistence layer—all implemented in safe, idiomatic Rust.

2

Chain: Domain Model

▷ What you will learn in this chapter

- How blocks and block headers encode the whitepaper's chain of hashes
- The UTXO model: why Bitcoin tracks *outputs*, not accounts
- How `TXInput`, `TXOutput`, and `Transaction` encode spending
- Where "unspent vs. spent" is tracked (the UTXO set, not the output itself)

In this section, we walk through the project's Rust types that represent Bitcoin's core domain objects. Our goal is to confidently answer "what does this type *mean* in whitepaper terms?" and "where is it used in the implementation?"

As we walk through the code, we use these files as our primary reference:

- `bitcoin/src/primitives/block.rs`
- `bitcoin/src/primitives/transaction.rs`
- `bitcoin/src/chain/utxo_set.rs` (for how outputs become "spendable")

2.1 Whitepaper Term → Rust Type Map

Table 2.1: Whitepaper concepts mapped to Rust types

Whitepaper Concept	Rust Type / Field
Block header	`BlockHeader`
Previous block hash	`BlockHeader.pre_block_hash`
Nonce	`BlockHeader.nonce`
Block hash	`BlockHeader.hash`
Transaction	`Transaction`
Transaction ID	`Transaction.id`
Input (spend)	`TXInput`
Output (new coin)	`TXOutput`
Locking condition	`TXOutput.pub_key_hash`
Signature	`TXInput.signature`

2.2 Domain Object Diagrams

The diagram below shows the *runtime relationship* between the main state buckets: the **blockchain** (append-only history), the **UTXO set** (derived "what is spendable right now" state), and **transactions** (which consume old outputs and create new ones).

```
                      Blockchain
↪

    Block 0 (Genesis)
      > Block 1
           > Block 2
                > Block N (Tip)



                              maintains derived state


                          UTXOSet
```

```
                        (txid,vout)    -> TXOutput
                        (txid,vout)    -> TXOutput
```

2.3 Key Code Reading Checklist

As we follow the Rust code, we should be able to answer these questions from the structs alone:

- Where does the txid come from? (Search for `Transaction::hash()`.)
- Where does the signature live? (Look at `TXInput.signature`.)
- What does an output lock to? (`TXOutput.pub_key_hash`.)
- Where is "unspent vs. spent" tracked? (Not on the output—in `UTXOSet`.)

2.4 Step-by-Step Code Walkthrough

Goal: build a correct mental model of what the core structs *mean*, and how the bytes flow between modules.

> i
>
> Source `bitcoin/src/primitives/block.rs`, `bitcoin/src/primitives/transaction.rs`, `bitcoin/src/chain/utxo_set.rs`

Step 1: Block and BlockHeader

In the Bitcoin whitepaper, a block is a **timestamped commitment** to two things: the previous block (so the history is tamper-evident by hash chaining) and a set of transactions (so the block anchors ownership transfers into the history).

In our implementation, `BlockHeader` is the smallest header we need to make those ideas concrete in code. In practice, the header matters because it defines the **proof-of-work input**: mining repeatedly hashes a byte sequence derived from the header (plus a transaction commitment) until it finds a valid `nonce`.

Those bytes are assembled from the header fields in `ProofOfWork::prepare_data(...)`, which is why `BlockHeader` contains exactly the fields it does. We cover the proof-of-work loop and the exact hashed byte layout in Section 9.6 (Block Lifecycle and Mining).

Listing 2: Block and BlockHeader struct layout (`block.rs`)

What the fields mean:

- `pre_block_hash`: link to the previous block (whitepaper §3/§4).
- `nonce` + `hash`: PoW search result (whitepaper §4).
- `timestamp`: ordering signal; also part of the PoW header bytes.
- `height`: convenience for iteration/selection; not a Bitcoin header field.

> ▷ **Project-specific note**
>
> In our implementation we do not use a **Merkle root** field. We instead concatenate transaction IDs and hash once using `Block::hash_transactions()`. This keeps the code readable but does not support Merkle proofs or SPV.

Creating a new block: in our implementation, `Block::new_block(pre_block_hash,` is the entry point that *defines* the header boundary. It takes the parent link and the transaction list as inputs, sets derived header fields such as `timestamp`, and then finalizes `nonce` and `hash` by running PoW.

Listing 3: `Block::new_block` — creating and mining a block

> ✓ **Checkpoint**
>
> `pre_block_hash` is the chain link. `nonce` and `hash` are filled by PoW (`ProofOfWork::run()`), not supplied by the caller. `height` lives in the header as a convenience (not a Bitcoin header field).

Step 2: Transaction, Inputs, and Outputs

Bitcoin is not an "account system." It is a system of **outputs**, and spending means consuming previous outputs as inputs.

> **i**
>
> Methods involved `Transaction::new_utxo_transaction()`, `Transaction::hash()`, `Transaction::sign()`

In this step, we do two things: (1) define what a transaction *is* by reading the struct fields, and (2) show how a spending transaction is *constructed* in code.

Listing 4: Transaction, TXInput, and TXOutput structs (`transaction.rs`)

They reveal the project's UTXO worldview: a transaction spends prior outputs by referencing an outpoint $(txid, vout)$ in `TXInput`, and it creates new spendable outputs in `TXOutput`.

- A spend is identified by an outpoint $(txid, vout)$: `TXInput.{txid, vout}`.
- Authorization is per-input: `TXInput.signature` + `TXInput.pub_key`.
- Outputs are locked by a pubkey-hash: `TXOutput.pub_key_hash` (no script engine).

How a spending transaction is constructed:

1. **Select spendable outputs**: query the UTXO set for outpoints locked to the sender.
2. **Create inputs**: for each selected outpoint, create a `TXInput`.
3. **Create outputs**: payment to recipient, plus optional change back to sender.
4. **Compute the txid**: hash a serialized copy with `id = []` (non-circular).
5. **Sign**: produce per-input signatures and store into `TXInput.signature`.

The concrete entry point is `Transaction::new_utxo_transaction(...)` (full implementation in Section 9.5: Transaction Lifecycle).

Step 3: History vs. Derived State (the UTXO Set)

UTXO (Unspent Transaction Output): a specific transaction output that is currently spendable—an output identified by an outpoint $(txid, vout)$ that has **not** been consumed by any later transaction input.

The **blockchain** is the append-only history (blocks and the transactions they contain). The **UTXO set** is derived state: a compact, query-friendly view of "what is spendable right now?" computed from that history.

We keep the UTXO set separate so the node can answer spendability questions efficiently without rescanning the entire chain every time it validates a transaction or connects a block.

Listing 5: UTXOSet struct (`utxo_set.rs`)

▷ **Note**

Spendability is not a field on `TXOutput`; it is tracked by membership in the UTXO database. This is the same design Bitcoin Core uses: the `chainstate` LevelDB stores only unspent outputs.

↪

See the full implementation The complete `find_spendable_outputs()` method, UTXO rebuild logic, and rollback handling are covered in Section 9.4 (UTXO Set).

Acronyms

This document is incomplete. The external file associated with the glossary 'acronym' (which should be called `main.acr`) hasn't been created.

Check the contents of the file `main.acn`. If it's empty, that means you haven't indexed any of your entries in this glossary (using commands like `\gls` or `\glsadd`) so this list can't be generated. If the file isn't empty, the document build process hasn't been completed.

Try one of the following:

- Add `automake` to your package option list when you load `glossaries-extra.sty`. For example:
  `\usepackage[automake]{glossaries-extra}`
- Run the external (Lua) application:
  `makeglossaries-lite.lua "main"`
- Run the external (Perl) application:
  `makeglossaries "main"`

  Then rerun LaTeX on this document.

  This message will be removed once the problem has been fixed.

Glossary

This document is incomplete. The external file associated with the glossary 'main' (which should be called `main.gls`) hasn't been created.

Check the contents of the file `main.glo`. If it's empty, that means you haven't indexed any of your entries in this glossary (using commands like `\gls` or `\glsadd`) so this list can't be generated. If the file isn't empty, the document build process hasn't been completed.

If you don't want this glossary, add `nomain` to your package option list when you load `glossaries-extra.sty`. For example:

```
\usepackage[nomain]{glossaries-extra}
```

Try one of the following:

- Add `automake` to your package option list when you load `glossaries-extra.sty`. For example:
  `\usepackage[automake]{glossaries-extra}`
- Run the external (Lua) application:
  `makeglossaries-lite.lua "main"`
- Run the external (Perl) application:
  `makeglossaries "main"`

Then rerun LaTeX on this document.

This message will be removed once the problem has been fixed.

Index

block header, 6

UTXO, 2

Colophon

This book was typeset using LaTeX with the `memoir` document class, configured for Crown Quarto format (7.44 × 9.69 inches).

Typography

The body text is set in *TeX Gyre Pagella* (a Palatino revival) at 11 pt with 1.05× leading. Section headings and navigational elements use *TeX Gyre Heros* (Helvetica family). Code listings and inline code use *TeX Gyre Cursor* (Courier family). Mathematical notation uses *Palatino Math* via the `mathpazo` package. Microtypographic refinements—character protrusion, font expansion, and tracking—are provided by the `microtype` package.

Color system

The book's visual identity is built around Rust Orange (`#B7410E`) for accent elements and Dark Teal (`#2C5F7C`) for structural headings. Eleven callout box families provide instant visual classification of notes, tips, warnings, checkpoints, and code references.

Code presentation

Source code is syntax-highlighted using the `minted` package with Pygments. Rust listings include line numbers and support inline LaTeX annotations via escape sequences. Manning-style numbered annotation circles connect code lines to explanatory notes below each listing.

Production tools

The PDF was produced with pdfLaTeX and `latexmk`. Diagrams were created with TikZ. The index was generated with `makeindex` and the glossary with `glossaries-extra`. Version control was managed with Git.

First Edition — 2026